

Guion de narración — SoK MCP + caso OPA

Duración estimada: 22–25 minutos + preguntas

Slide 1 — Título

Buenas. Lo que voy a presentar hoy es un trabajo de investigación en curso, parte de mi doctorado, sometido en modalidad double-blind a SaTML 2027. No es un trabajo cerrado — les voy a mostrar exactamente hasta dónde llega la evidencia y dónde empieza la especulación, porque esa distinción es, en sí misma, parte de lo que quiero que se lleven de esta clase.

El eje de la charla tiene dos partes: primero, qué encontramos auditando 89 servers reales de Model Context Protocol; segundo, por qué Open Policy Agent — OPA — es un buen caso de estudio para pensar cómo se debería gobernar la autorización en este tipo de sistemas, aunque en este trabajo no construimos ningún prototipo con OPA. Se va a entender por qué es importante esa aclaración a medida que avancemos.

Slide 2 — ¿Por qué importa esto ahora?

Model Context Protocol es el estándar que publicó Anthropic para que un agente basado en un LLM pueda invocar herramientas externas: bases de datos, APIs, el filesystem, wallets de criptomonedas, historias clínicas. Cualquier cosa que un desarrollador quiera exponerle a un agente.

El punto de seguridad es simple de enunciar y difícil de resolver: cada *tool* que un server MCP expone es, en los hechos, una nueva superficie de autorización. Y quien decide *cuál* tool invocar no es un usuario tipeando un comando explícito — es un modelo de lenguaje interpretando una intención en lenguaje natural, a veces influenciado por contenido que ni el usuario ni el operador controlan directamente, como el texto de una página web que el agente resumió.

Entonces la pregunta que atraviesa todo este trabajo es: cuando un agente puede llamar a una tool, ¿quién decidió qué tiene permitido hacer, y cómo se hizo cumplir esa decisión en el código real?

Slide 3 — El estado de la literatura

Antes de meternos en los datos, vale la pena ubicar dónde está parada la literatura de seguridad de MCP hoy. Hay básicamente dos géneros.

Uno son los *surveys* de amenazas: catalogan clases de ataque — tool poisoning, prompt injection indirecto, confusión de alcance — pero no miran código de servers reales, son ejercicios de sistematización sobre papers y reportes.

El otro son los escaneos automatizados a gran escala: hay un estudio que escaneó 67.000 servers, otro con casi 1.900. Miden prevalencia de bugs conocidos con muy buena cobertura, pero necesariamente de forma superficial por repositorio — un scanner no lee el diseño de gobernanza de un sistema, detecta patrones de código.

Ninguno de los dos géneros responde la pregunta que le importa a alguien que tiene que decidir cómo construir o auditar uno de estos servers: cuando un proyecto *sí* intentó gobernar lo que un agente puede hacer, ¿cómo se ve eso cuando funciona bien? ¿Y cuando falla, específicamente por qué falla?

Slide 4 — Este trabajo

Eso es lo que hicimos: una auditoría manual, no automatizada, de 89 repositorios MCP públicos y distintos entre sí — 91 pasadas de auditoría en total porque un par de repos se revisaron dos veces por distintos motivos que documentamos en el paper. Quince rondas, estratificadas por dominio: infraestructura, bases de datos, identidad, pagos, salud, CRM, y así.

Cada hallazgo lo clasificamos por nivel de evidencia, porque no es lo mismo decir "leí el código y creo que esto es explotable" que "ejecuté la construcción completa contra una reconstrucción aislada y esto es lo que devolvió". Usamos tres tiers: lectura de código, ejecución con el efecto final mockeado, y ejecución completa contra un entorno reconstruido — nunca contra un target en producción.

La idea metodológica de fondo es que un scanner automatizado tira un número. Nosotros leímos código, ronda por ronda, con un ser humano — yo — revisando y aprobando cada hallazgo antes de que entrara al corpus.

Slide 5 — Hallazgo central

Y acá está el resultado que más nos sorprendió, y el que sostiene toda la charla de hoy: menos del 10% de los repositorios tiene un bug de código explotable — inyección, SSRF, ese tipo de cosas clásicas.

La falla dominante no es un error de programación. Es una decisión de autorización que nunca se tomó. Encontramos servers que exponen 1 a 1 toda la API subyacente de una plataforma — tanto las operaciones de lectura como las de control administrativo — como tools de MCP, sin ningún criterio de reducción por mínimo privilegio, todo bajo una única credencial compartida.

Quiero que se detengan en esa frase: no falta código. Falta una decisión de diseño. Eso cambia completamente qué tipo de solución tiene sentido buscar — y es la bisagra hacia la segunda mitad de la

charla.

Slide 6 — Taxonomía de tres capas

Para poder hablar de esto con precisión en vez de "está mal gobernado", propusimos una taxonomía de tres capas más un eje ortogonal.

Capa uno: exposición del *control plane* — qué parte de la API subyacente termina siendo una tool invocable. La falla característica es el mapeo 1 a 1 sin reducción.

Capa dos: ejecución y confianza — con qué identidad se ejecuta la tool, y si hay algún gate de confirmación humana antes de una mutación de alto impacto. La falla característica es una credencial compartida y estática, sin ese gate.

Capa tres: señalización de riesgo a nivel de protocolo — si el server le dice al cliente MCP qué tan peligrosa es una tool, usando las anotaciones nativas del protocolo. La falla característica es simplemente no usarlas.

Y el eje ortogonal, que atraviesa a las tres capas: cuando sí se intenta construir un límite, ¿está bien construido? Porque puede existir la intención y fallar la implementación — eso lo vamos a ver en el caso de salud en un momento.

Slide 7 — El marco teórico ya existe

Ahora, nada de esto es conceptualmente nuevo. El marco teórico ya existe en la literatura de seguridad de sistemas, y lo que hace este trabajo es mostrar que se instancia, de forma muy concreta, en MCP.

El "confused deputy problem" de Norman Hardy, de 1988: un programa que ejecuta con más autoridad de la que le confirió quien lo invocó. Acá, el agente LLM es exactamente ese *deputy* — actúa entre la intención de un usuario y la autoridad completa de una tool, y puede terminar usando esa autoridad completa aunque la intención original fuera mucho más acotada.

El "indirect prompt injection" de Greshake y colegas, de 2023, es el complemento del lado de la entrada: contenido que el agente ingiere como si fuera un dato — una página web, un mensaje — puede redirigir qué hace el agente con su autoridad.

Y el tercer concepto, que es donde vamos a llegar con OPA: ABAC, control de acceso basado en atributos, y la separación entre punto de decisión de política y punto de aplicación de política — PDP y PEP. Open Policy Agent con su lenguaje Rego es el ejemplo canónico de esa arquitectura. Es el vocabulario estándar para el patrón de "política externalizada, fail-closed" que vamos a ver que aparece, de forma espontánea, en nuestro corpus.

Slide 8 — Caso 1: sandbox escape en salud

Vamos a los casos concretos, porque son los que hacen tangible todo esto.

Encontramos un server MCP de historias clínicas que carga el registro FHIR completo de un paciente en memoria — condiciones, medicación, notas clínicas, resultados de laboratorio — y expone una tool que evalúa JavaScript arbitrario provisto por quien llama, para hacer análisis ad hoc sobre ese registro. La implementación usa el módulo `vm` de Node.js como "sandbox".

El problema es que la propia documentación de Node.js dice, textualmente, que el módulo `vm` no es un mecanismo de seguridad y que no hay que usarlo para correr código no confiable. Nosotros reconstruimos exactamente esa construcción de sandbox en un script aislado, contra un objeto sintético — nunca contra el server real en producción — y con código que ni siquiera hacía referencia al registro del paciente, logramos una referencia viva al objeto `process` del host. En microsegundos, sin necesidad de evadir ningún timeout.

Y la tool no tenía ninguna anotación de riesgo, igual que las otras cuatro tools del mismo server. Esto no es "no se intentó contener nada". Es: se declaró explícitamente una intención de sandboxing, y el primitivo elegido no cumple esa garantía. Es, para nosotros, el caso más limpio del corpus para argumentar que auditar gobernanza no puede quedarse en "¿existe un mecanismo?" — tiene que preguntar "¿el mecanismo elegido realmente entrega la garantía que el código asume que entrega?".

Slide 9 — Caso 2: la tool que bypassa la autorización

El segundo caso es distinto en naturaleza. Un server MCP de Salesforce expone catorce tools; siete están correctamente anotadas como de solo lectura y pasan por la REST API estándar, que respeta seguridad a nivel de campo y reglas de compartición.

Pero hay una octava tool que ejecuta código Apex arbitrario — el lenguaje propietario de Salesforce — en lo que se llama "modo sistema": ahí no aplican las reglas de seguridad que sí respetan las tools de lectura del mismo server, salvo que el propio código Apex las invoque explícitamente, cosa que no se le puede exigir a un string arbitrario que viene de afuera. Y la descripción de la tool literalmente invita a usarla como fallback general.

Clasificamos esto como el ejemplo más claro del corpus de una tool cuyo propósito explícito es saltar el modelo de autorización que gobierna a todas las demás tools del mismo server — sin ninguna anotación, sin ningún gate que la distinga de las siete tools de lectura de al lado. No es un problema de "falta un candado en una puerta". Es una puerta construida específicamente para no tener candado.

Slide 10 — Cuando la gobernanza sí funciona

Ahora el contrapeso, porque el trabajo no es solo un catálogo de fallas. Cuando la gobernanza está bien hecha, tampoco es un patrón único: armamos un catálogo de diez patrones de mitigación, observados en código real y desplegado, no propuestos en abstracto.

Lo interesante es que cuatro de esos diez patrones aparecen de forma independiente en proyectos que nunca tuvieron contacto entre sí — misma solución, descubierta por separado. Eso es evidencia de que responden a una necesidad de diseño real, no a una moda o a copiar de un mismo tutorial.

Voy a nombrar el que nos interesa hoy: el patrón número tres, "decisión de política externalizada y fail-closed". Y quiero mostrarles el código real detrás, porque es la bisagra hacia la parte de OPA.

Slide 11 — El patrón 3 en código real

Este patrón lo encontramos en un server de wallet de Lightning Network. Antes de ejecutar un pago, hay un webhook de pre-ejecución que consulta un endpoint de política externo, separado del código que efectivamente mueve el dinero. Y el diseño es fail-closed: si ese endpoint no responde, o devuelve algo que el server no reconoce, la decisión por default es denegar.

Miren lo que implica esto arquitectónicamente: la pregunta "¿puede este caller ejecutar este pago?" no vive adentro del handler que ejecuta el pago. Vive afuera, en un componente separado, consultado antes de actuar.

Eso es, literalmente, una separación PDP-PEP hecha a mano. El autor de este server probablemente no pensó "voy a implementar el patrón arquitectónico que Open Policy Agent formaliza" — llegó a la misma solución resolviendo el mismo problema. Y esto no es una idea de laboratorio ni un diagrama en un paper: ya apareció, de forma convergente, en producción, resolviendo pagos reales.

Slide 12 — Por qué OPA es un buen caso (parte 1)

Ahora sí, la pregunta central de esta charla: ¿por qué OPA es un buen caso para pensar la autorización en MCP? Y quiero que las razones queden ancladas en lo que acabamos de ver en el corpus, no en que "OPA es una herramienta popular".

Primera razón: el hallazgo central de todo el trabajo es un problema de política, no de código. "Nadie tomó la decisión" es exactamente el vacío que un policy decision point externalizado está diseñado para llenar — la política se declara aparte de la lógica de negocio, en un lugar donde se puede leer, versionar y auditar, en vez de vivir implícita, o directamente ausente, adentro de cada handler de cada tool.

Segunda razón: MCP ya expone gratis el punto de decisión correcto. Cada invocación de una tool es un mensaje JSON-RPC estructurado — nombre de la tool, argumentos, identidad de quien llama —

interceptable en un único lugar, antes de que se ejecute nada. Eso es exactamente el *enforcement point* que un PEP necesita. No hay que inventar dónde interceptar; el protocolo ya lo define.

Slide 13 — Por qué OPA es un buen caso (parte 2)

Tercera razón: el protocolo ya trae metadata de riesgo lista para ser el input de una política. Las anotaciones nativas de MCP — si una tool es de solo lectura, si es destructiva, si es idempotente, si opera en un mundo abierto — son exactamente el tipo de atributo que un motor ABAC como Rego evalúa de forma natural. Y nuestra capa tres de la taxonomía muestra que casi nadie las usa de forma sistemática: solo un 11% del corpus. Un PDP externalizado puede forzar que esas anotaciones se respeten, incluso si el desarrollador del server las omitió o las usó mal.

Cuarta razón, y muy práctica: es retrofit-able sin tocar el server. Porque MCP es un protocolo de red — no una librería que hay que embeber en el código — un PEP puede vivir como un proxy o un sidecar delante de cualquiera de los 89 servers que auditamos, incluidos los que fallaron en capa uno o capa dos, sin reescribir una sola línea de su código fuente.

Slide 14 — Honestidad metodológica

Y acá viene la parte que, como docentes de seguridad, probablemente valoran más que cualquier otra cosa de esta charla: qué es lo que NO hicimos.

No construimos ningún prototipo con OPA. No hay proxy en Go, no hay integración con Rego, no hay benchmark de latencia contra un dataset de ataques. OPA aparece en este trabajo en dos lugares nada más: como vocabulario arquitectónico en la sección de trabajo relacionado, y como el patrón que ya observamos, de forma completamente informal, en código real — el caso de la wallet de Lightning que acabamos de ver.

Construir y evaluar ese sistema completo sería un paper distinto — un paper de sistemas, no un SoK — y eso implica meses de ingeniería real que no salen de otra ronda de auditoría de repositorios. La distinción entre "tengo la fundamentación teórica resuelta" y "tengo la arquitectura resuelta" no es un detalle menor: es una disciplina que tuvimos que ejercer activamente durante todo el proyecto, porque es muy tentador, cuando ves el mismo patrón aparecer tres veces, convencerte de que ya lo probaste en la práctica. No lo probamos. Lo observamos.

Slide 15 — La severidad no es solo código

Un matiz importante antes de cerrar esta parte: ni siquiera un policy engine perfectamente implementado resuelve todo el problema, si la política no conoce la semántica de la plataforma que hay detrás de la tool.

Un `delete` en un server de contabilidad puede ser un soft-delete completamente reversible; un `DeleteObject` de un bucket de S3, no. Un server de trading puede correr por default contra un entorno de paper trading simulado, y solo pasar a dinero real con un opt-in explícito del operador — la misma tool sin restricciones tiene un riesgo categóricamente distinto antes y después de ese opt-in. Y un server de búsqueda puede ejecutar el fetch desde su propio proceso, o proxearlo hacia la infraestructura del vendor — eso cambia contra qué red apunta un SSRF exitoso, incluso si el código es idéntico.

La conclusión práctica es que una política tipo OPA necesita, como input, esta semántica de plataforma — no alcanza con el nombre de la tool y sus argumentos. Es un límite real, y lo dejamos explícito en el paper en vez de simplificarlo.

Slide 16 — Posicionamiento

Brevemente, cómo se para este trabajo frente a la literatura más cercana en los últimos seis meses: hay un SoK previo sobre MCP, un estudio a escala de registro con 67.000 servers aceptado en DSN 2026, un estudio empírico con 1.899 servers, y una taxonomía de ubicación de defensas por capa arquitectónica.

Ninguno de los cuatro combina auditoría empírica original con un catálogo de patrones de mitigación positivos, y ninguno ejecuta un proof-of-concept completo como el del sandbox escape. Nuestra escala es dos órdenes de magnitud menor que el estudio más grande — pero la profundidad por repositorio es correspondientemente mayor.

Slide 17 — Límites

Y los límites del trabajo, dichos sin vueltas, porque forman parte de cómo hay que leer cualquier resultado empírico en seguridad.

Es una muestra de conveniencia, no un crawl sistemático de un registro — no podemos afirmar representatividad estadística. Es un solo revisor — yo — sin segundo codificador ni verificación adversarial independiente. Y el proceso de disclosure responsable está incompleto: de cinco hallazgos críticos, dos ya fueron reportados formalmente y están esperando respuesta del mantenedor, y los otros tres todavía no tienen un canal de disclosure funcional al momento de escribir esto.

Slide 18 — Conclusión

Para cerrar: la falla dominante que encontramos es arquitectónica, no sintáctica. Es ausencia de decisión, no error de programación.

Este corpus alimenta una línea de trabajo más amplia sobre hardening continuo de autorización en sistemas agénticos, organizada como un framework de doble loop. Un red loop que no se conforme con preguntar "¿existe el límite declarado?", sino que también pregunte "¿está bien construido?" — el caso del sandbox de salud muestra por qué la primera pregunta sola no alcanza. Y un blue loop que trate "construimos el mecanismo" y "el mecanismo está activo por default y resiste la construcción exacta que un atacante probaría" como dos afirmaciones separadas, ambas necesarias.

Un policy decision point externalizado, en el estilo de OPA y Rego, es el bloque natural para ese blue loop — no como una conclusión ya demostrada por este trabajo, sino como el próximo paso de investigación, con evidencia empírica real que lo justifica: ya lo vimos aparecer, de forma independiente, en producción.

Slide 19 — Preguntas

Gracias. Quedo abierto a preguntas — y si alguien quiere profundizar en la parte metodológica, en los tiers de evidencia, o en la discusión ética del disclosure, con gusto entramos ahí también.

Notas para quien presenta

- **Tono:** es un trabajo en curso, sometido a revisión — decirlo explícitamente ayuda a que la audiencia académica confíe en el resto de las afirmaciones.
- **Si preguntan "¿por qué no usaron OPA directamente en la auditoría?":** la respuesta honesta es que el corpus se armó para *caracterizar* qué gobernanza existe, no para *construir* gobernanza nueva — son dos preguntas de investigación distintas, y mezclarlas hubiera diluido ambas.
- **Si preguntan por qué no otro motor de políticas (Cedar, Casbin, etc.):** aclarar que el argumento no es "OPA es superior técnicamente" — es que la arquitectura PDP/PEP que OPA formaliza es la que el patrón 3 del catálogo ya instancia de forma artesanal. Cualquier motor que respete esa separación calza con la misma evidencia.
- **Tiempo:** si hay que recortar, las slides 8–9 (casos) y 12–13 (por qué OPA) son el núcleo; 16–17 (posicionamiento/límites) son las más comprimibles sin perder el argumento central.